# MODUL 10 DASAR-DASAR DETEKSI OBYEK

## A. TUJUAN

- Mahasiswa dapat memahami konsep deteksi objek
- Mahasiswa mampu memahami beberapa metode yang dapat digunakan dalam proses deteksi objek
- Mahasiswa mampu memahami konsep Feature Matching
- Mahasiswa mampu memahami konsep face detection
- Mahasiswa dapat mengimplementasikan beberapa metode dalam proses deteksi objek dan proses Feature Matching dan Face detection menggunakan Python pada Google Colab

## B. ALAT DAN BAHAN

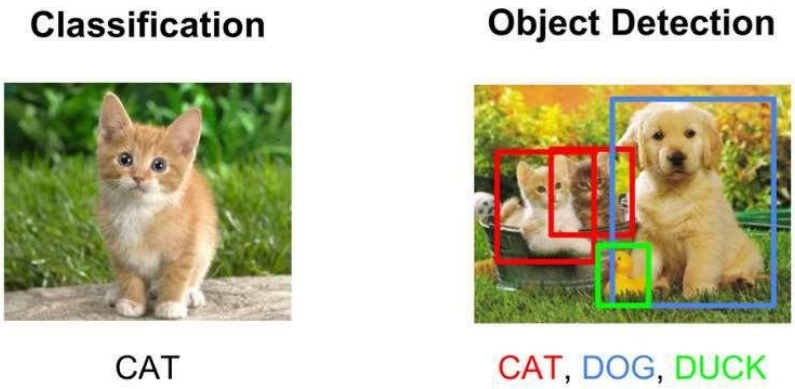
1. PC/LAPTOP
2. Github
3. *Google Colaboratory*

## C. DASAR TEORI

### C.1 Dasar Deteksi Obyek

#### C.1.1 Konsep Deteksi Objek

Deteksi objek (object detection) merupakan salah satu teknik visi komputer (computer vision) yang berfungsi untuk mengidentifikasi dan menemukan objek dalam sebuah gambar atau video. Secara khusus, proses deteksi objek akan menggambar kotak pembatas di sekitar objek yang terdeteksi, sehingga kita dapat mengetahui lokasi objek tersebut pada gambar atau video. Tujuan dari deteksi objek adalah untuk mendeteksi semua *instance* objek dari kelas yang telah diketahui, seperti orang, mobil atau wajah pada gambar. Sebuah gambar umumnya hanya memuat sedikit objek, tetapi jumlah kemungkinan lokasi dan skala objek tersebut sangat banyak. Berikut ini adalah ilustrasi dari konsep deteksi objek:




Dalam sistem computer vision, pendeteksian objek merupakan tahap pertama yang dilakukan karena hasil dari deteksi objek dapat memungkinkan untuk memperoleh informasi lebih lanjut mengenai objek yang terdeteksi dan lokasi objek tersebut. Setelah objek terdeteksi (misalnya, wajah), kita dapat memperoleh informasi lebih lanjut, antara lain: (i) mengenali contoh spesifik (misalnya, untuk mengidentifikasi subjek dari wajah yang terdeteksi), (ii) melacak objek pada urutan gambar (misalnya, untuk melacak wajah dalam video), dan (iii) mengekstrak informasi lebih lanjut tentang objek (misalnya, untuk menentukan jenis kelamin subjek). Beberapa contoh implementasi deteksi objek antara lain:

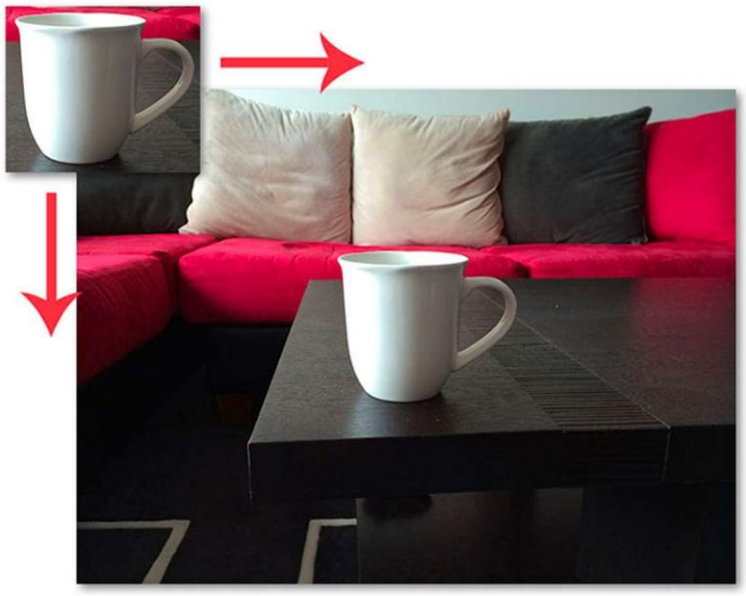
1. Video surveillance (CCTV), model deteksi objek mampu melacak banyak orang sekaligus, secara real-time
2. Crowd counting (Penghitungan kerumunan), untuk area padat pengunjung seperti taman hiburan, mall, atau alun-alun kota, deteksi objek dapat membantu pemilik bisnis dan pemerintahan kota untuk mengukur dengan lebih efektif berbagai jenis lalu lintas, dan beberapa rencana seperti jam operasional toko, shift karyawan, dan pengalokasian sumber daya.
3. Anomaly detection, misal di bidang pertanian, model pendeteksian objek dapat secara akurat mengidentifikasi dan menemukan lokasi potensi penyakit tanaman, sehingga memungkinkan petani untuk mendeteksi ancaman terhadap hasil panen mereka yang tidak dapat dilihat dengan mata telanjang.
4. Self-driving cars, deteksi objek merupakan teknik dasar pada mobil tanpa pengemudi. Sistem ini harus dapat mengidentifikasi, menemukan, dan melacak objek di sekitar mobil agar dapat bergerak di jalan raya dengan aman dan efisien.

### C.1.2 Template Matching

Pencocokan template (template matching) merupakan bentuk yang paling dasar dari deteksi objek. Template matching adalah metode untuk mencari dan menemukan lokasi gambar template pada gambar yang lebih besar. Diperlukan dua gambar untuk menerapkan metode template matching, yaitu:

- Source image (I): gambar masukan yang diharapkan cocok dengan template.
- Template image (T): potongan objek yang akan dicari pada source image.

Untuk menemukan template di source image sumber, dilakukan pergeseran template dari kiri ke kanan dan dari atas ke bawah:



Di setiap lokasi (x, y), sebuah metrik dihitung untuk merepresentasikan seberapa "baik" atau "buruk" kecocokan tersebut. Perhitungan koefisien korelasi dilakukan untuk menentukan seberapa mirip intensitas piksel dari dua potongan gambar tersebut. Untuk setiap lokasi ***T*** pada ***I***, metrik hasil perhitungan akan disimpan dalam matriks hasil ***R***. Setiap koordinat (x, y) pada source image akan berisi elemen di matriks hasil ***R***:



Lokasi cerah dari matriks hasil R menunjukkan nilai kecocokan yang terbaik, sedangkan daerah gelap menunjukkan korelasi yang sangat kecil antara gambar sumber dan template.

OpenCV mengimplementasikan template matching menggunakan fungsi **matchTemplate()**. Terdapat 6 metode yang tersedia pada matchTemplate(), yaitu:

1. method=TM_SQDIFF

$$R(x,y) = \sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2$$

2. method=TM_SQDIFF_NORMED

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

3. method=TM_CCORR

$$R(x,y) = \sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))$$

4. method=TM_CCORR_NORMED

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') \cdot I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

5. method=TM_CCOEFF

$$R(x,y) = \sum_{x',y'}(T'(x',y') \cdot I'(x+x',y+y'))$$

6. method=TM_CCOEFF_NORMED

$$R(x,y) = \frac{\sum_{x',y'}(T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$$

Dokumentasi template matching pada OpenCV dapat dilihat pada link berikut: https://docs.opencv.org/3.4/de/da9/tutorial_template_matching.html

### C.1.3 Edge Detection

Deteksi tepi (Edge detection) adalah operasi yang dijalankan untuk mendeteksi garis tepi (edges) yang membatasi dua wilayah citra yang memiliki tingkat kecerahan yang berbeda. Deteksi tepi sebuah citra digital merupakan proses untuk mencari perbedaan intensitas yang menyatakan batas-batas suatu objek (sub-citra) dalam keseluruhan citra digital yang dimaksud. Suatu titik (x,y) dikatakan sebagai tepi dari suatu citra bila titik tersebut memiliki perbedaan intensitas yang tinggi terhadap piksel tetangga. Deteksi tepi banyak dipakai untuk mengidentifikasi suatu objek dalam sebuah gambar. Terdapat 3 metode edge detection yang disediakan oleh OpenCV, yaitu:

1. Sobel Edge Detection

   Sobel adalah salah satu detektor tepi yang paling umum digunakan. Filter Sobel bekerja dengan menghitung gradien intensitas citra pada setiap piksel dalam sebuah citra. Saat menggunakan filter ini, gambar dapat diproses dalam arah X dan Y secara terpisah atau bersamaan. Sobel didasarkan pada konvolusi gambar dalam arah horizontal dan vertical menggunakan filter 3x3, berikut:

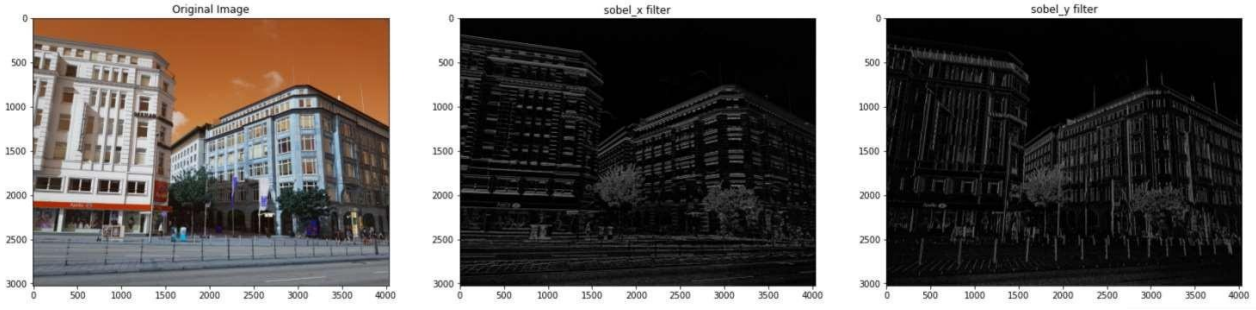
| | | |
|---|---|---|
| -1 | 0 | +1 |
| -2 | 0 | +2 |
| -1 | 0 | +1 |

Gx

| | | |
|---|---|---|
| +1 | +2 | +1 |
| 0 | 0 | 0 |
| -1 | -2 | -1 |

Gy

Contoh hasil dari sobel edge detection adalah sebagai berikut:



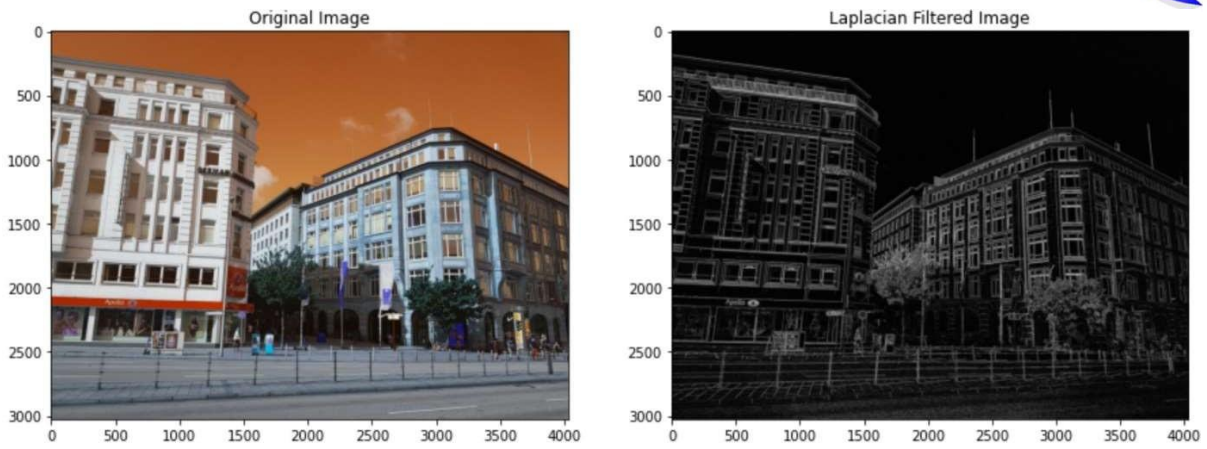

2. Laplacian Edge Detection

Kernel 3x3 yang umumnya digunakan pada Laplacian Edge Detection adalah sebagai berikut:

| | | |
|---|---|---|
| 0 | −1 | 0 |
| −1 | 4 | −1 |
| 0 | −1 | 0 |

| | | |
|---|---|---|
| −1 | −1 | −1 |
| −1 | 8 | −1 |
| −1 | −1 | −1 |

Metode ini sangat sensitif terhadap noise, maka dari itu kita menghaluskan gambar terlebih dahulu dengan Gaussian filter sebelum menerapkan Laplacian. Contoh hasil dari laplacian edge detection adalah sebagai berikut:




3. Canny Edge Detection

   Canny Edge Detection merupakan metode yang paling umum digunakan dan paling efektif. Metode ini bekerja dengan 4 tahap, yaitu:

   1. Menghaluskan gambar dengan filter Gaussian untuk mengurangi noise
   2. Menghitung gradien menggunakan salah satu operator gradien Sobel
   3. Mengekstraksi titik tepi: Non-maximum suppression
   4. Menghubungkan dan thresholding: Hysteresis

Canny edge detection didasarkan pada konsep bahwa intensitas gambar bernilai tinggi pada bagian tepi. Permasalahan pada konsep ini adalah jika gambar memiliki noise, maka noise tersebut juga akan terdeteksi sebagai tepi. Sehingga langkah pertama dalam Canny edge detection adalah menghilangkan noise, salah satu cara adalah menerapkan filter Gaussian untuk menghaluskan gambar sebelum melanjutkan pemrosesan.

Langkah kedua dalam proses deteksi tepi dengan Canny adalah komputasi gradien yang dilakukan dengan menghitung tingkat perubahan intensitas (gradien) pada gambar di sepanjang arah gradien. Citra yang telah dihaluskan pada langkah pertama kemudian difilter dengan kernel Sobel, baik horizontal maupun vertikal untuk mendapatkan turunan pertama dalam arah horizontal (Gx) dan vertikal (Gy). Dari kedua gambar tersebut, dapat dicari gradien tepi dan arah untuk setiap piksel sebagai berikut:

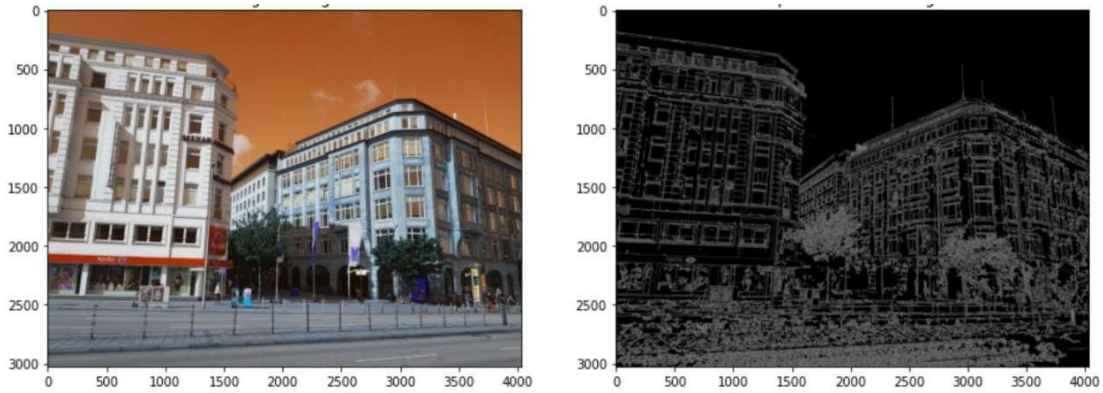
$$Edge_Gradient\ (G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle\ (\theta) = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

Intensitas gambar mencapai titik tertinggi pada bagian tepi. Pada kenyataan, intensitas tidak memuncak hanya pada satu piksel, melainkan pada beberapa piksel yang bersebelahan. Di setiap lokasi piksel, Canny edge detection akan membandingkan piksel dan memilih local maximal di 3X3 ketetanggaan ke arah gradien. Proses ini dikenal sebagai Non-maximum suppression.

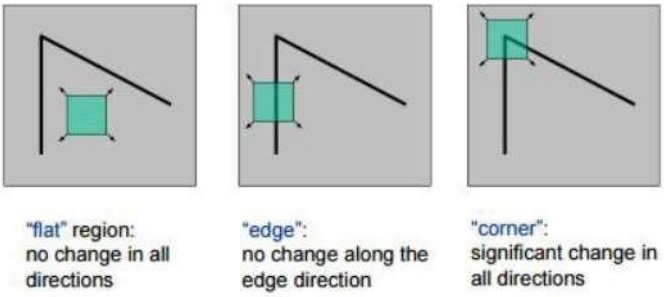
Langkah ketiga menghasilkan garis tepi berukuran tipis tetapi patah. Langkah terakhir adalah memperbaiki / menyambungkan tepi-tepi yang patah tersebut dengan menggunakan teknik hysteresis thresholding. Pada hysteresis thresholding, terdapat dua ambang batas: ambang batas tinggi dan ambang rendah (high and low threshold). Setiap piksel dengan nilai gradien lebih tinggi dari high threshold secara otomatis disimpan sebagai tepi. Sedangkan piksel dengan nilai gradien di antara high threshold dan low threshold ditangani dengan dua cara. Piksel akan diperiksa apakah memiliki kemungkinan koneksi dengan tepi; piksel yang tersambung disimpan, sedangkan piksel yang tidak tersambung dibuang. Piksel dengan gradien lebih rendah dari low threshold akan dibuang secara otomatis.

Contoh hasil dari canny edge detection adalah sebagai berikut:




### C.1.4 Corner Detection

Sudut dapat diartikan sebagai persimpangan dua sisi. Titik sudut tidak bisa didefinisikan pada piksel tunggal, karena di sana hanya terdapat pada satu gradien di setiap titik. Gradien adalah arah perubahan intensitas kecerahan dalam suatu citra. Sudut adalah fitur penting dalam sebuah gambar, dan umumnya disebut sebagai interest point yang tidak berubah apabila dilakukan translasi, rotasi, dan iluminasi.




Terdapat 2 metode corner detection yang disediakan oleh OpenCV, yaitu:

1. **Harris corner detection**

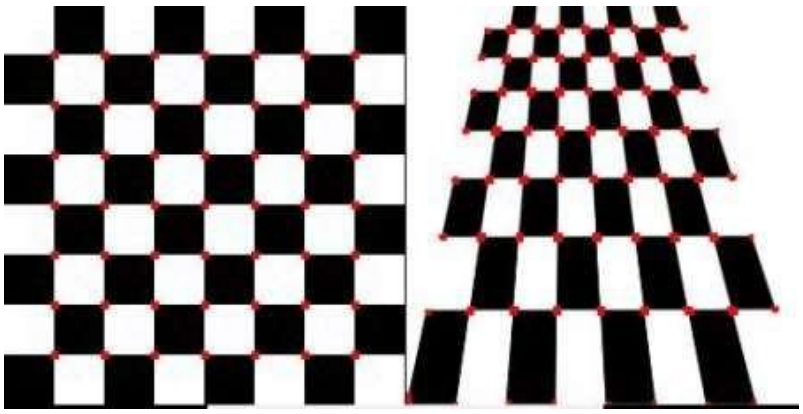
   Harris Corner Detection adalah sistem pendeteksi sudut yang sering digunakan karena mampu menghasilkan nilai yang konsisten pada citra yang mengalami rotasi, penskalaan, variasi pencahayaan, maupun memiliki banyak noise pada gambar. Deteksi sudut dengan metode Harris didasarkan pada variasi intensitas sinyal. Variasi intensitas yang besar menunjukkan keberadaan sudut pada citra. Cara kerja metode Harris adalah dengan menemukan perbedaan intensitas untuk perpindahan (u, v) ke segala arah, dengan persamaan berikut:

$$E(u,v) = \sum_{x,y} \underbrace{w(x,y)}_{\text{window function}} \underbrace{[I(x+u,y+v)}_{\text{shifted intensity}} - \underbrace{I(x,y)}_{\text{intensity}}]^2$$

   Window function dapat berupa filter Gaussian yang memberi bobot pada piksel di bawah jendela. Fungsi E (u, v) ini harus dimaksimalkan untuk deteksi sudut. Pada OpenCV, metode yang digunakan adalah:
   cv2.cornerHarris(src, dest, blockSize, kSize,
   freeParameter, borderType) dengan parameter:

   - Src: citra input
   - Dest: citra untuk menyimpan hasil Harris detector, dengan ukuran sama seperti citra input
   - blockSize: ukuran ketetanggaan
   - ksize: kernel size untuk operator Sobel
   - freeParameter: parameter bebas untuk Harris detector
   - borderType: metode pixel extrapolation

Contoh hasil Harris corner detection adalah sebagai berikut:



Dokumentasi lengkap dapat dibaca pada link berikut:

https://docs.opencv.org/3.4/dc/d0d/tutorial_py_features_harris.html

## 2. Shi-Tomasi Corner Detection

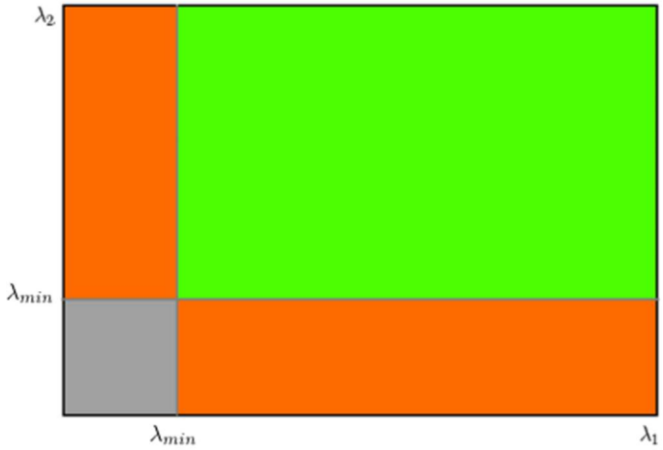
Shi-Tomasi Corner Detection menggunakan dasar bahwa sudut dapat dideteksi dengan mencari perubahan signifikan ke segala arah. Misal terdapat sebuah window kecil pada gambar kemudian memindai/scan seluruh gambar untuk mencari sudut. Menggeser window kecil ini ke segala arah akan menghasilkan perubahan besar pada tampilan, jika window tersebut terletak di sudut.
Pada metode Harris, penilaian sebuah titik dikatakan sudut atau bukan dilakukan dengan memberikan nilai R

$$R = det(M) - k(\mathrm{Tr}(M))^2$$

Shi-Tomasi mengajukan ide untuk penilaian R hanya dengan menentukan nilai Eigen dari M saja

$$R = \min(\lambda_1, \lambda_2)$$

Jika nilai R lebih besar daripada nilai threshold, maka titik tersebut dianggap sebagai sudut. Jika diplot di ruang λ1
− λ2, akan menjadi seperti berikut:




Dari gambar tersebut dapat dilihat bahwa hanya jika λ1 dan λ2 di atas nilai minimum, λmin dianggap sebagai sudut (area berwarna hijau).

OpenCV memiliki fungsi, cv.goodFeaturesToTrack(). Fungsi ini akan menemukan N sudut terkuat pada gambar dengan metode Shi-Tomasi. Citra input harus berupa gambar grayscale. Pengaturan
/ parameter dari metode ini antara lain: (a) jumlah sudut yang ingin ditemukan, (b) tingkat kualitas, yang merupakan nilai antara 0-1, yang menunjukkan kualitas minimum dari sudut, dan
(c) jarak euclidean minimum antar sudut yang terdeteksi. Dengan parameter-parameter tersebut, fungsi akan menemukan sudut-sudut pada gambar. Semua sudut di bawah tingkat kualitas akan ditolak. Sudut yang tersisa akan diurutkan berdasarkan kualitas dalam urutan menurun. Kemudian fungsi akan mengambil sudut terkuat pertama, membuang semua sudut terdekat dalam kisaran jarak minimum dan mengembalikan N sudut terkuat.

Dokumentasi lengkap dapat dibaca pada link berikut:

https://docs.opencv.org/master/d4/d8c/tutorial_py_shi_tomasi.html

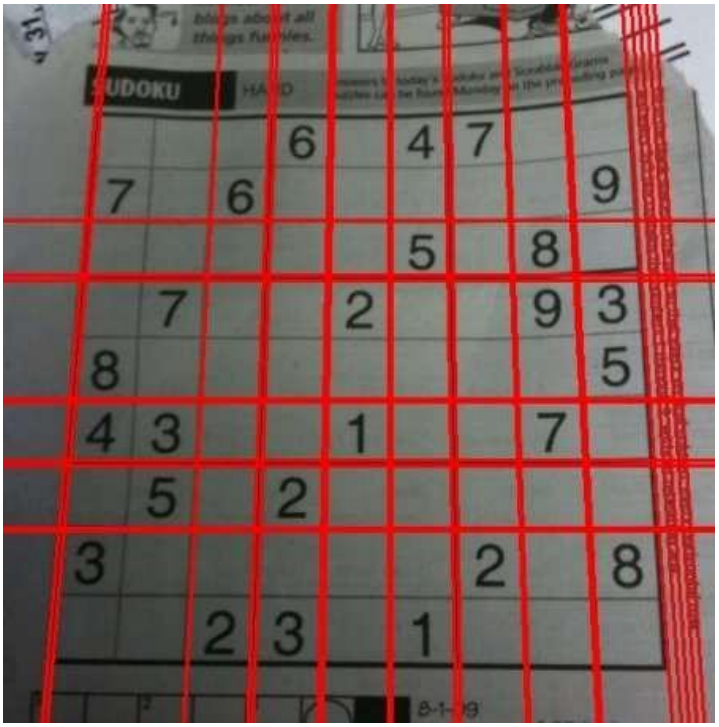
### C.1.5 Grid Detection

Grid detection dapat dilakukan dengan metode Hough Transform. Hough Transform adalah teknik populer untuk mendeteksi bentuk apa pun, jika bentuk tersebut dapat direpresentasikan dalam bentuk matematika. Hough Transform dapat mendeteksi bentuk bahkan jika bentuk tersebut rusak atau sedikit terdistorsi.

Tahapan untuk melakukan grid detection adalah sebagai berikut:

1. Menerapkan Canny Edge Detection
2. Dilasi gambar tepi (Canny bisa menemukan kedua tepi pemisah pada grid sebagai tepi yang berbeda, dilatasi dapat membuat kedua tepi ini bergabung lagi)
3. Erosi (setelah proses dilasi, garis border menjadi tebal sehingga Hough akan mendeteksi banyak garis)
4. Menerapkan HoughLines dengan fungsi cv2.HoughLines() pada OpenCV
5. Menggabungkan garis yang mirip (similar)

Contoh hasil Hough Transform adalah sebagai berikut:




Dokumentasi lengkap mengenai HoughTransform dapat dibaca pada link berikut:

https://docs.opencv.org/4.x/d6/d10/tutorial_py_houghlines.html

### C.1.6 Contour Detection

Kontur adalah kurva tertutup yang menghubungkan semua titik kontinu yang memiliki beberapa warna atau intensitas. Kontur mewakili bentuk objek yang ditemukan dalam gambar. Deteksi kontur adalah teknik yang berguna untuk analisis bentuk serta deteksi dan pengenalan objek. Kontur adalah kumpulan abstrak dari titik dan segmen

yang sesuai dengan bentuk objek pada gambar. Kita dapat memanipulasi kontur dalam program kita seperti menghitung jumlah kontur, memakai kontur untuk mengkategorikan bentuk objek, atau memotong objek dari gambar (segmentasi gambar).

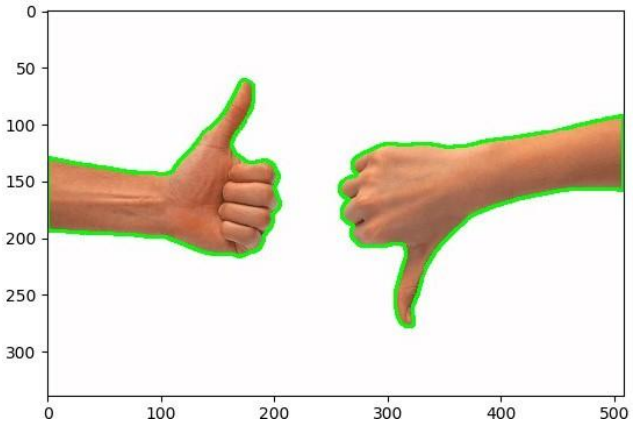
Untuk akurasi yang lebih baik, berikut ini merupakan tahapan untuk mendeteksi kontur pada gambar:

1. Ubah citra input menjadi citra biner (bisa didapat dengan mengimplementasikan thresholding atau canny edge detection).
2. Menemukan kontur menggunakan fungsi findContours() pada OpenCV.
3. Menggambarkan kontur pada citra input menggunakan fungsi drawContours().

Fungsi findContours menerima tiga buah parameter, yaitu :

- Parameter pertama adalah citra masukan yang akan dianalisis
- Parameter kedua menunjukkan metode retrieval. Metode retrieval tersebut dapat digunakan untuk melihat hierarcy dari kontur dalam sebuah citra. Hierarcy menunjukkan hubungan parent-child. Parents merupakan shape/contour yang lebih luar dibandingkan dengan child atau dengan kata lain, parents merupakan shape/contour yang melingkupi child.
- Parameter ketiga menunjukkan metode yang digunakan untuk proses approximation dari shape. Methode tersebut dapat berupa metode cv2.CHAIN_APPROX_SIMPLE atau metode cv2.CHAIN_APPROX_NONE. Pada flag cv2.CHAIN_APPROX_SIMPLE, class cv2.findContours akan mengembalikan seminimum mungkin jumlah koordinat yang menunjukkan contours sedangkan pada flag cv2.CHAIN_APPROX_SIMPLE, class cv2.findContours akan mengembalikan sebanyak mungkin jumlah koordinat yang menunjukkan contours.

Contoh hasil contour detection adalah sebagai berikut:




Dokumentasi lengkap dapat dibaca pada link berikut:

https://docs.opencv.org/master/d4/d73/tutorial_py_contours_begin.html

## C.2 Feature Matching dan Face Detection

### C.2.1 Konsep Feature Matching

Feature matching melakukan ekstraksi fitur penting dari sebuah citra menggunakan ide dasar dari corner, edge, dan contour detection. Selanjutnya sistem menghitung distance untuk mencari kecocokan antara image sumber dengan image template. Dengan cara ini, anda tidak perlu lagi memakai template yang persis sama dengan objek pada image sumber. Jika objek pada citra template tidak berukuran sama dengan citra sumber, deteksi tetap bisa dilakukan dengan baik. Pada praktikum kali ini akan dicobakan 2 metode Feature Matching yaitu:

1. Brute-Force Matching menggunakan ORB Descriptor (Oriented FAST and Rotated BRIEF)
2. Brute-Force Matching menggunakan SIFT Descriptor dan Test Ratio (Scale-Invariant Feature Transform)

ORB adalah berasal dari "OpenCV Labs". Algoritma ini diajukan oleh Ethan Rublee, Vincent Rabaud, Kurt Konolige dan Gary R. Bradski dalam artikel mereka berjudul "*ORB: An efficient alternative to SIFT or SURF*" pada tahun 2011. Seperti tertulis pada judul artikel tersebut, ORB adalah alternatif pengganti dari SIFT dan SURF dalam hal biaya komputasi, kinerja matching dan terutama adalah paten. SIFT dan SURF dipatenkan, sehingga kita harus membayar lisensi untuk memakai kedua algoritma tersebut, sedangkan ORB bebas dipakai tanpa biaya.

ORB merupakan perpaduan antara detektor keypoint FAST dan deskriptor BRIEF, dengan banyak modifikasi untuk meningkatkan kinerja. Sebagai langkah awal, FAST

digunakan untuk menemukan keypoint, lalu Harris Corner detection digunakan untuk mencari N-point tertinggi. Akan tetapi FAST tidak menghitung orientasi keypoint. Modifikasi dilakukan agar algoritma ini invariant terhadap orientasi. Penjelasan lebih detil dapat dilihat pada dokumentasi OpenCV: https://docs.opencv.org/3.4/d1/d89/tutorial_py_orb.html

ORB Descriptor telah disediakan pada library Opencv cv.ORB(). Berikut adalah code untuk menunjukkan penggunaan ORB Descriptor. Pada praktikum ini anda diminta untuk menggunakan file reeses_puff dan cereals yang telah disediakan. Perhatikan pada 2 gambar tersebut, reeses puff memiliki resolusi yang lebih besar daripada reeses puff pada image cereals.

```
def display(img,cmap='gray'):
  fig = plt.figure(figsize=(12,10))
  ax = fig.add_subplot(111)
  ax.imshow(img,cmap='gray')
```

```
reeses = cv.imread('/content/drive/MyDrive/Polinema/Kuliah/PCVK/Images/facedet/reeses_puffs.png'
display(reeses)
```




```
cereals = cv.imread('/content/drive/MyDrive/Polinema/Kuliah/PCVK/Images/facedet/many_cereals.jpg
display(cereals)
```



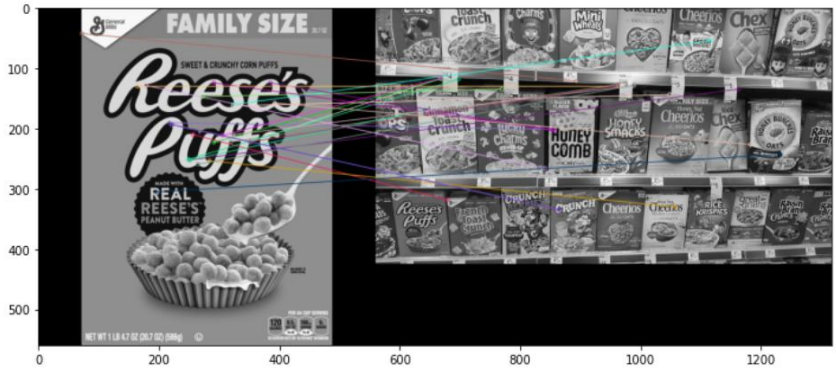
```
orb = cv.ORB_create()
kp1,des1 = orb.detectAndCompute(reeses,None)
kp2,des2 = orb.detectAndCompute(cereals,None)
bf = cv.BFMatcher(cv.NORM_HAMMING,crossCheck = True)
matches = bf.match(des1,des2)
single_match = matches[0]
single_match.distance
```

```
71.0
```

```
len(matches)
```

```
139
```

```
matches = sorted(matches,key=lambda x:x.distance)
reeses_match = cv.drawMatches(reeses,kp1,cereals,kp2,matches[:25],None,flags=2)
display(reeses_match)
```




Perhatikan hasil pencocokan diatas. Dengan memilih 25 titik yang paling cocok, hasil dari pencocokan pada kasus diatas tidak akurat sama sekali.

Pada pembahasan sebelumnya tentang Harris corner detection misal, metode-metode

tersebut bersifat rotation-invariant, yaitu corner tetap terdeteksi dengan baik walaupun image terotasi. Hal ini jelas sekali, karena corner akan tetap berupa corner walaupun objek dirotasikan. Berbeda kasus dengan scaling. Corner bisa jadi bukan lagi corner jika citra discaling. Perhatikan gambar berikut. Corner pada citra kecil dengan window berukuran kecil akan berupa garis jika diperbesar dan menggunakan window berukuran sama. Hal ini menunjukkan bahwa Harris corner bukan algoritma yang scale-invariant.

Pada tahun 2004, D. Lowe dari University of British Columbia, menemukan algoritma baru, Scale Invariant Feature Transform (SIFT) dalam artikel berjudul “Distinctive Image Features from Scale-Invariant Keypoints”. Algoritma ini mengekstrak keypoint dan menghitung deskriptor. Artikel ini mudah dipahami dan dianggap sebagai materi terbaik yang tersedia di SIFT.

Terdapat 4 tahap yang terjadi pada algoritma SIFT. Penjelasan lebih detil dapat anda baca pada dokumentasi OpenCV:

https://docs.opencv.org/master/da/df5/tutorial_py_sift_intro.html . Berikut akan ditunjukkan penggunaan algoritma SIFT pada image reese puffs dan cereal. Hasil yang

```
!pip install opencv-contrib-python==4.4.0.44
```

```
sift = cv.xfeatures2d.SIFT_create()
kp1,des1 = sift.detectAndCompute(reeses,None)
kp2,des2 = sift.detectAndCompute(cereals, None)
bf = cv.BFMatcher()
matches = bf.knnMatch(des1,des2,k=2)

good = []

#Makin kecil jarak makin cocok
#ratio match1 < 75% Match2
for match1,match2 in matches:
  #jika jarak match1 lebih kecil dari jarak 75% match2
  #descriptor disimpan
  if match1.distance < 0.75*match2.distance:
    good.append([match1])

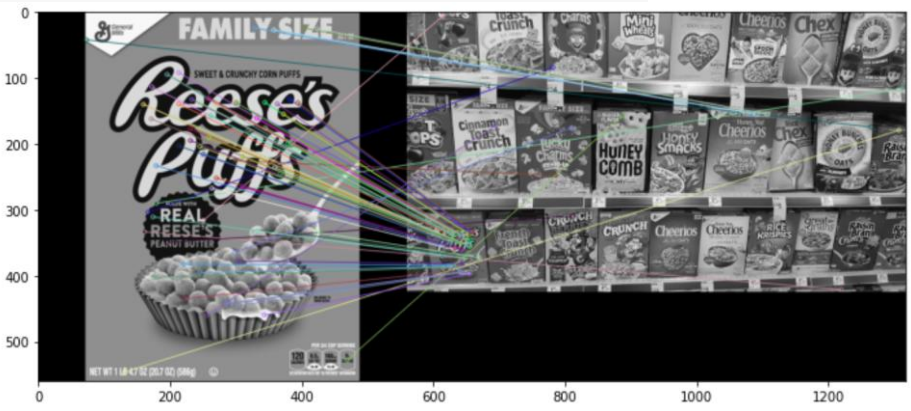
len(good)
```

```
79
```

```
len(matches)
```

```
1501
```

```
sift_matches = cv.drawMatchesKnn(reeses,kp1,cereals,kp2,good,None,flags=2)
display(sift_matches)
```



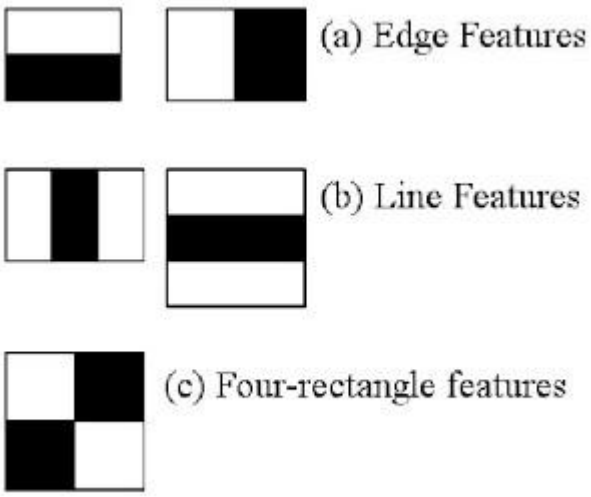

didapatkan juga jauh lebih baik jika dibandingkan dengan ORB Descriptor. Untuk

menggunakan SIFT descriptor, anda harus menggunakan OpenCV versi 4.4.0.44 ke atas. Google Colab saat ini sudah menyertakan OpenCV 4.x, sehingga umumnya SIFT langsung dapat dipakai. Jalankan instalasi lewat pip (pip install opencv-python) hanya jika cv.SIFT_create() masih menghasilkan error.

**C.2.2 Konsep Face Detection**

Pada materi ini akan dijelaskan cara kerja face detection menggunakan Haar Cascades, yaitu komponen kunci dari framework deteksi object Viola-Jones. Metode ini dapat mendeteksi wajah dengan cepat pada sebuah image dan mengetahui lokasi wajah dengan tepat. Metode ini digunakan sebagai tahap awal dalam mengenali wajah (Facial Recognition).

Pada tahun 2001, Paul Viola dan Michael Jones mempublikasikan metode face detection berdasarkan pada konsep sederhana dari beberapa fitur kunci. Ide lain yang digunakan adalah melakukan perhitungan awal dari integral image untuk mempercepat waktu komputasi. Perhatikan tipe-tipe fitur utama yang diajukan oleh Viola dan Jones.




Tiap fitur adalah merupakan nilai tunggal yang didapatkan dengan mengurangkan rata-rata pixel pada area kotak putih dengan rata-rata pixel pada area kotak hitam.

| | | | |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |

| | | | |
|---|---|---|---|
| 0 | 0.1 | 0.8 | 1 |
| 0.3 | 0.1 | 0.7 | 0.8 |
| 0.1 | 0.2 | 0.8 | 0.8 |
| 0.2 | 0.2 | 0.8 | 0.8 |

mean(area terang) – mean(area gelap).

Sum([0.8, 1, 0.7, 0.8, 0.8, 0.8, 0.8, 0.8]) = 6.5

Sum([0, 0.1, 0.3, 0.1, 0.1, 0.2, 0.2, 0.2]) = 1.2

Mean = 6.5 / 8 = 0.8125

Mean = 1.2 / 8 = 0.15

Delta = 0.8125 – 0.15 = 0.6625

Nilai delta tersebut kemudian di threshold sesuai kebutuhan kita. Semakin tinggi nilai delta (==1) maka akan semakin cocok fitur tersebut sebagai edge. Jika dimisalkan diberi nilai Threshold = 0.5, sedangkan nilai delta adalah 0.6625, maka fitur tersebut dianggap sebagai edge.

Menghitung nilai delta untuk keseluruhan citra akan membuat komputasi menjadi sangat mahal. Algoritma Viola-Jones mengajukan metode integral image.

| | | | | | |
|---|---|---|---|---|---|
| 31 | 2 | 4 | 33 | 5 | 36 |
| 12 | 26 | 9 | 10 | 29 | 25 |
| 13 | 17 | 21 | 22 | 20 | 18 |
| 24 | 23 | 15 | 16 | 14 | 19 |
| 30 | 8 | 28 | 27 | 11 | 7 |
| 1 | 35 | 34 | 3 | 32 | 6 |

| | | | | | |
|---|---|---|---|---|---|
| 31 | 33 | 37 | 70 | 75 | 111 |
| 43 | 71 | 84 | 127 | 161 | 222 |
| 56 | 101 | 135 | 200 | 254 | 333 |
| 80 | 148 | 197 | 278 | 346 | 444 |
| 110 | 186 | 263 | 371 | 450 | 555 |
| 111 | 222 | 333 | 444 | 555 | 666 |

15 + 16 + 14 + 28 + 27 + 11 =

101 + 450 – 254 – 186 = 111

Algoritma ini juga dijalankan dengan cepat dengan menggunakan **cascade classifier**. **Cascade classifier menghitung nilai fitur image secara seri, mulai dari fitur sederhana yang telah ditunjukkan pada bagian sebelumnya.** Misalkan dilakukan ekstraksi fitur edge terlebih dahulu, dan jika tidak termasuk sebagai fitur edge, maka sistem tidak menghitung fitur line dan rectangle, lalu bergeser ke area pencarian lain.

Langkah awal yang dilakukan adalah dengan mengubah citra masukan ke dalam Grayscale. Fitur awal yang digunakan sebagai pencarian adalah edge feature yang menunjukkan keberadaan mata dan pipi. Jika citra tidak menunjukkan keberadaan fitur ini, sistem menganggap tidak ada wajah pada citra tersebut. Jika terindikasi ada, maka sistem melanjutkan pencarian ke fitur lain, seperti nose bridge (bagian tengah hidung), lips (bibir), alis. Pencarian terus dilakukan menggunakan cascading, yang berarti akan terdapat ribuan fitur yang terkumpul. Secara teori, pendekatan ini dapat digunakan pada berbagai objek dan deteksi. Sebagai contoh, dapat digunakan untuk mendeteksi mata, bibir, hidung dan lain sebagainya. Atau dapat juga digunakan untuk mendeteksi objek

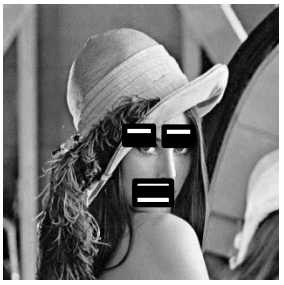
lain seperti mobil, motor, pesawat, dan lain sebagainya. Kelemahan dari algoritma ini adalah kebutuhan akan referensi dataset yang besar untuk mendapatkan fitur yang diinginkan. Saat ini fitur-fitur deteksi objek menggunakan Viola-Jones dapat ditemukan dengan mudah. OpenCV juga menyediakan fitur-fitur tertentu yang sering dipakai oleh banyak peneliti. Fitur-fitur tersebut disediakan OpenCV dengan format XML.



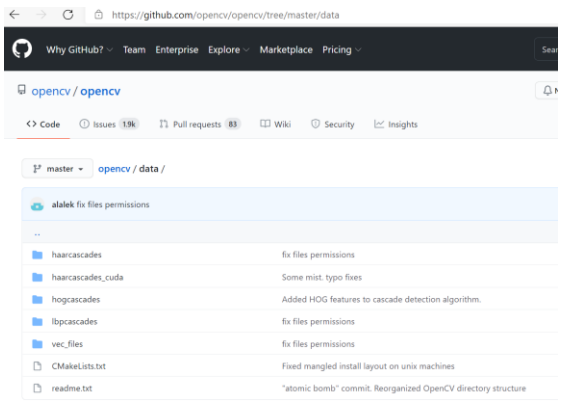


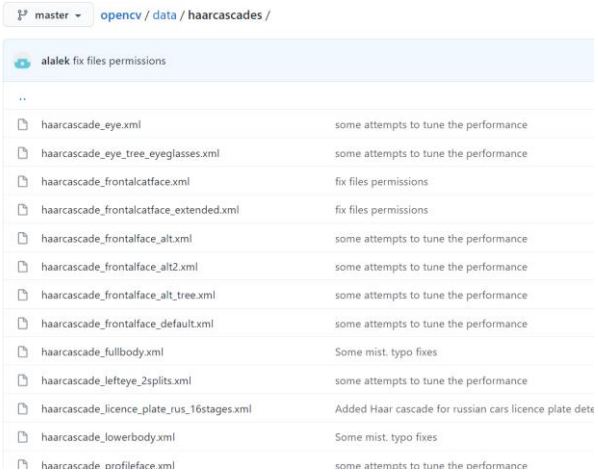











Percobaan berikut akan digunakan untuk melakukan face detection menggunakan OpenCV dan algoritma Viola Jones. Dokumentasi lengkap dapat dilihat pada OpenCV Documentation:

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_objdetect/py_face_detection/py_face_detection.html

Beberapa images telah disediakan pada folder (/images/facedet). Untuk pretrained features juga telah disediakan pada folder (/images/haarcascades). Silahkan akses secara langsung pada folder tersebut. Fitur pretrained yang digunakan adalah

```
cascade_wajah = cv.CascadeClassifier('/content/drive/MyDrive/Polinema/Kuliah/PCVK/Images/haarcascades/haarcascade_frontalface_alt.xml')
#sesuai posisi folder Haarcascade masing-masing

prabowo = cv.imread('/content/drive/MyDrive/5 Perkuliahan JTI/Materi Perkuliahan/MK PCVK/PCVK 2025/Images/facedet/prabowo.jpg',0)
roi_wajah = cascade_wajah.detectMultiScale(prabowo)
for(x,y,w,h) in roi_wajah:
  cv.rectangle(prabowo,(x,y),(x+w,y+h),(255,255,255),1)
plt.figure(figsize = (10,10))
plt.imshow(prabowo, cmap='gray')
```

“haarcascade_frontalface_alt.xml”. fungsi OpenCV yang digunakan adalah menggunakan “detectMultiScale”. Jika ditemukan wajah, maka akan digambarkan rectangle pada lokasi wajah tersebut menggunakan “rectangle”.




Face tracking dengan live camera mudah dilakukan pada Python dengan OpenCV, sayang sekali untuk Google Colab tidak bisa secara langsung menggunakan Camera yang ada pada komputer personal atau laptop, sehingga dibutuhkan code JavaScript untuk dapat mengakses kamera. code snippet untuk akses kamera telah tersedia juga pada Menu kiri Google Colab.

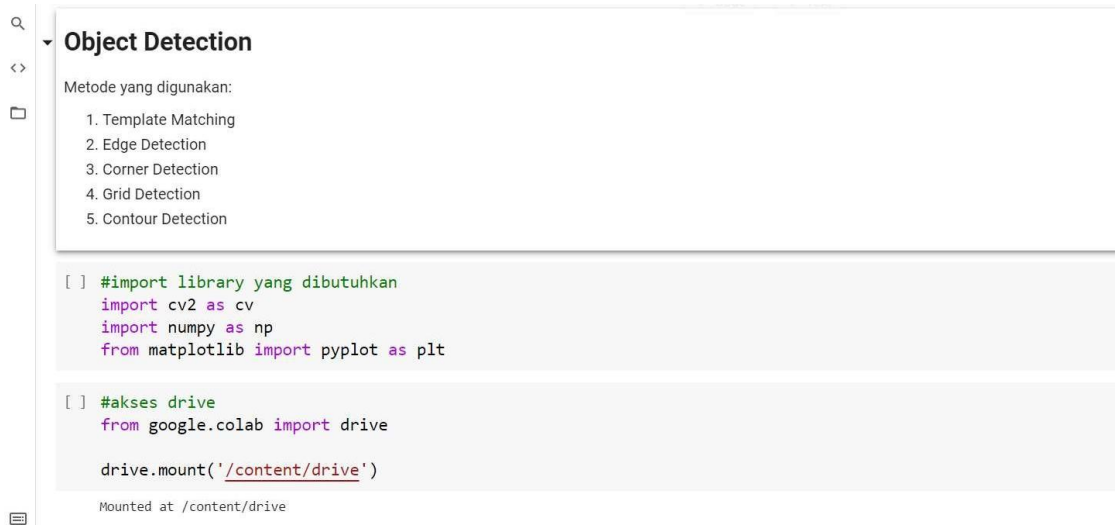
## D. TUGAS PRAKTIKUM

**Catatan**: Untuk gambar pada praktikum ini menggunakan gambar pada link berikut:

https://drive.google.com/drive/folders/1M30vcqgPBD1PHQpqFUtxWdtiMrwSpijC?usp=drive_link

## D1. Praktikum Deteksi Obyek Dasar

1 Buka https://colab.research.google.com/. Setelah dipastikan bahwa google Colab terhubung dengan Github Anda, buat notebook baru dan beri nama "Week10.ipynb". Kemudian import beberapa library dan akses folder yang ada di Drive Anda dengan cara sebagai berikut.



**Object Detection**

Metode yang digunakan:

1. Template Matching
2. Edge Detection
3. Corner Detection
4. Grid Detection
5. Contour Detection

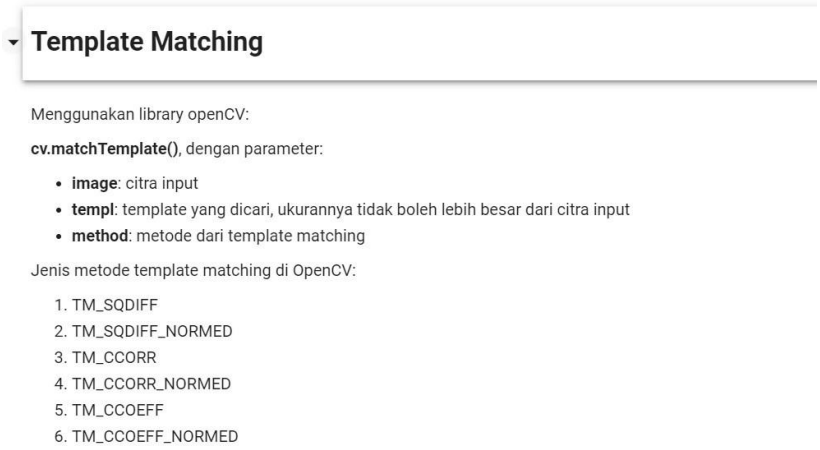
```
[ ] #import library yang dibutuhkan
    import cv2 as cv
    import numpy as np
    from matplotlib import pyplot as plt
```

```
[ ] #akses drive
    from google.colab import drive

    drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

2 Implementasikan 6 metode template matching pada OpenCV dengan menggunakan gambar cats_and_bunnies.jpg dan cat2_templatejpg.jpg sebagai citra template.



**Template Matching**

Menggunakan library openCV:

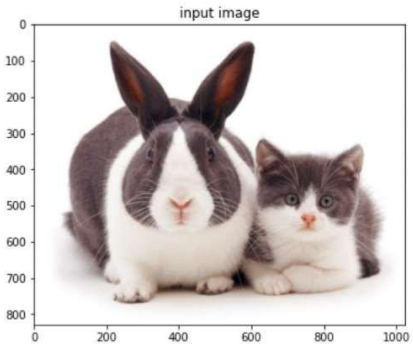
**cv.matchTemplate()**, dengan parameter:

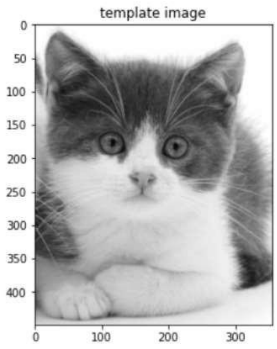
- **image**: citra input
- **templ**: template yang dicari, ukurannya tidak boleh lebih besar dari citra input
- **method**: metode dari template matching

Jenis metode template matching di OpenCV:

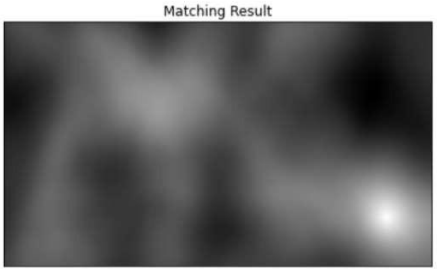
1. TM_SQDIFF
2. TM_SQDIFF_NORMED
3. TM_CCORR
4. TM_CCORR_NORMED
5. TM_CCOEFF
6. TM_CCOEFF_NORMED

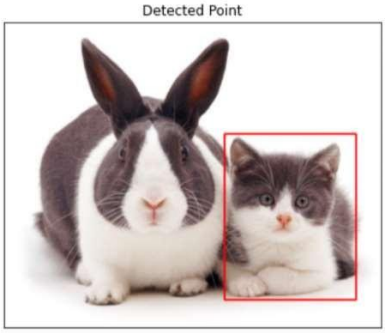
Sehingga menghasilkan luaran seperti berikut:



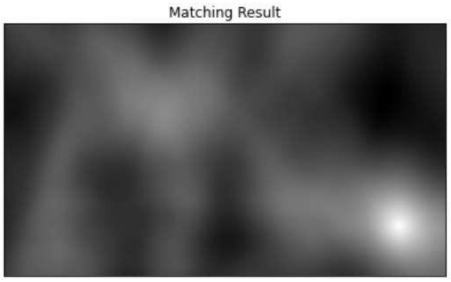



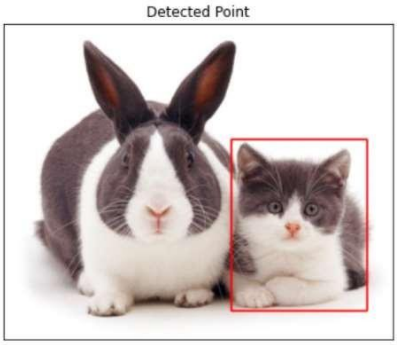

cv.TM_CCOEFF



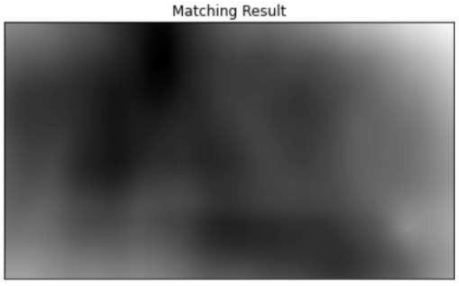



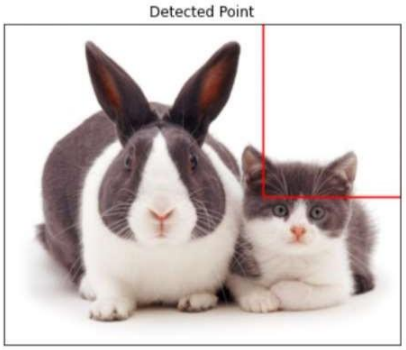

cv.TM_CCOEFF_NORMED



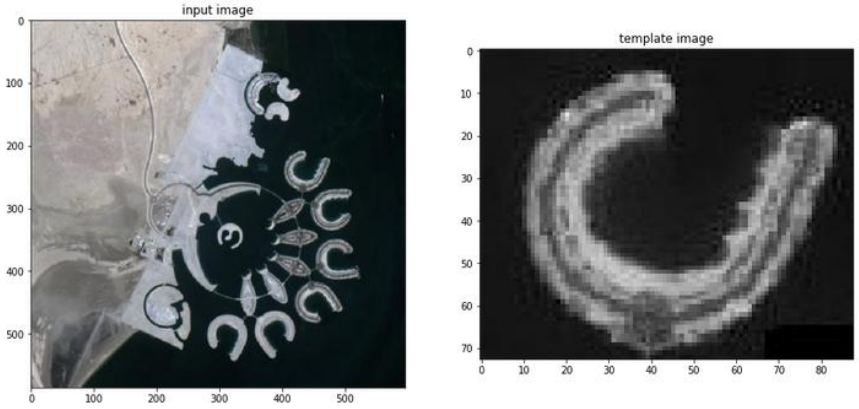

cv.TM_CCORR

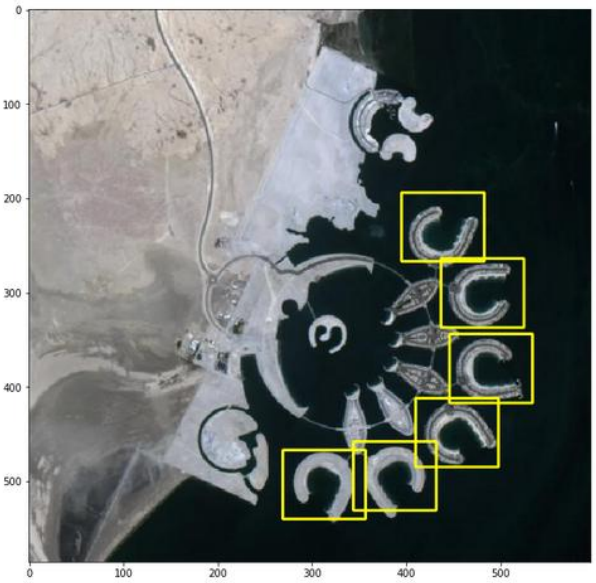





3 Implementasikan konsep template matching tanpa menggunakan library OpenCV untuk multiple object, menggunakan gambar bahrain.jpg untuk citra masukan dan bahrain-template.jpg sebagai citra template, sehingga menghasilkan output sebagai berikut:

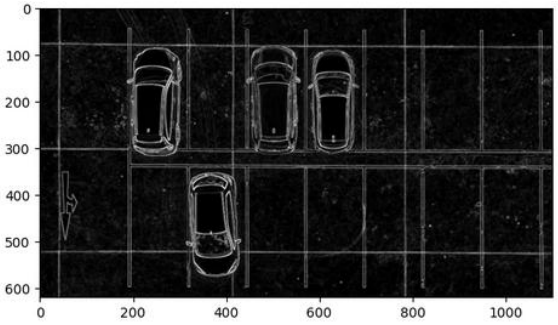




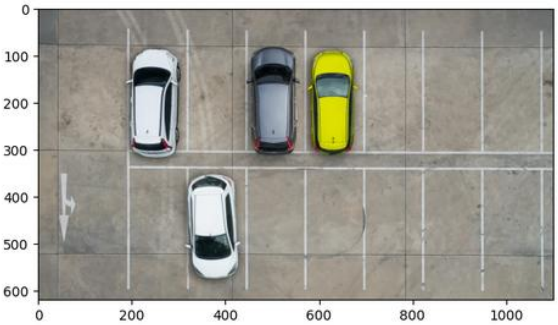
4 Implementasikan metode Sobel Edge Detection, Canny Edge Detection, dan Laplacian Edge Detection pada OpenCV dengan menggunakan gambar car-park.jpg, sehingga menghasilkan luaran sebagai berikut:

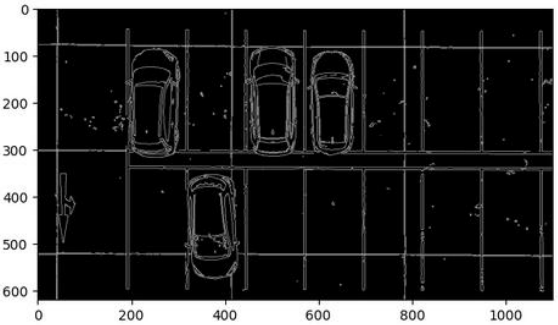
a. Sobel Edge Detection



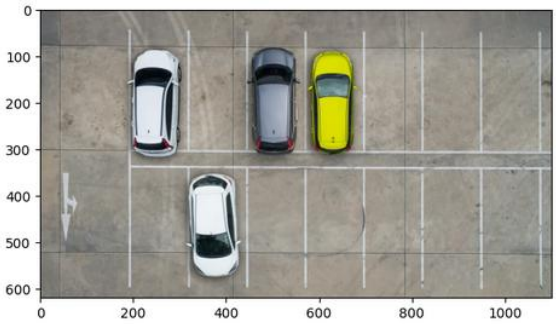


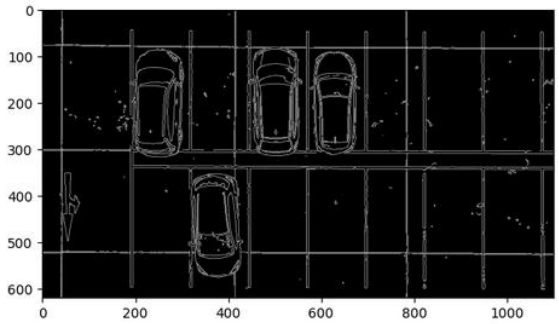
b. Canny Edge Detection





c. Laplacian Edge Detection



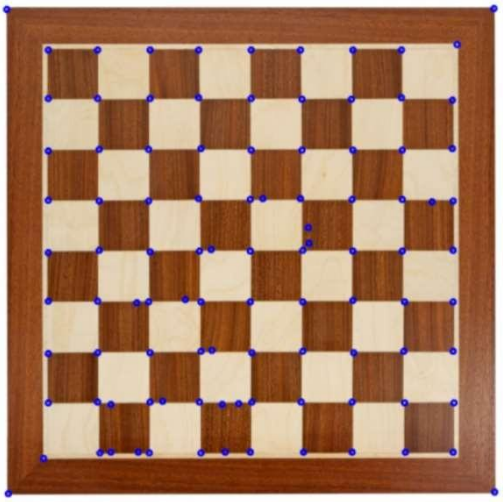




5 Implementasikan metode Harris Corner Detection dan Shi-Tomasi Detection pada OpenCV dengan menggunakan gambar chess-board.jpg, sehingga menghasilkan luaran sebagai berikut:

a. Harris Corner Detection

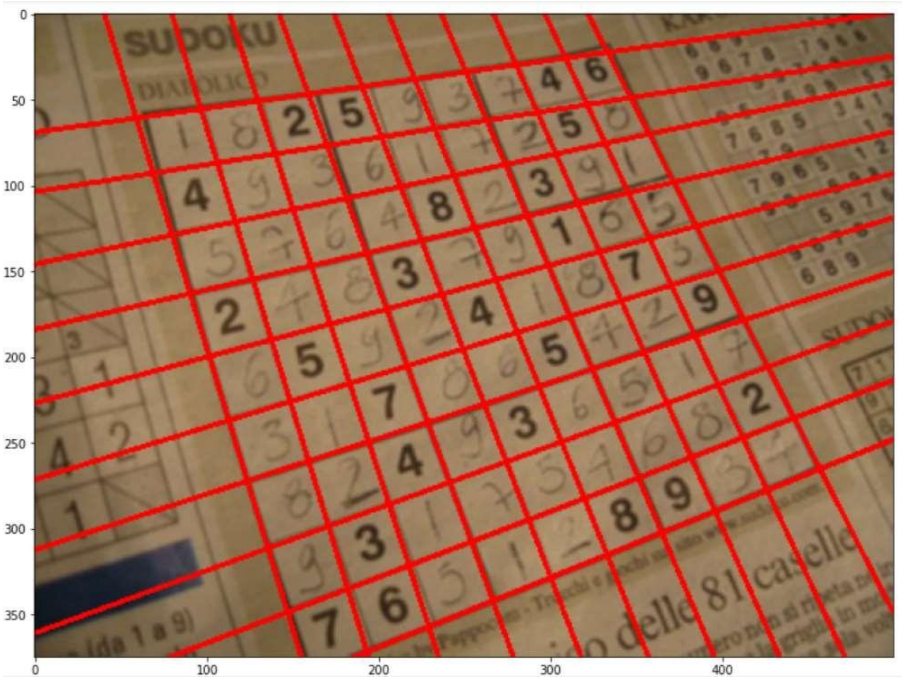


b. Shi-Tomasi Detection



6 Implementasikan metode Hough Transform pada OpenCV dengan

menggunakan gambar sudoku.jpg. Tahapan proses grid detection sesuai yang terdapat pada ulasan teori, sehingga menghasilkan luaran sebagai berikut:



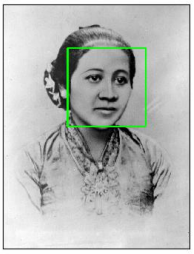
7. Implementasikan fungsi findContours() pada OpenCV untuk contour detection dengan menggunakan gambar laptop.jpg, sehingga menghasilkan luaran sebagai berikut:



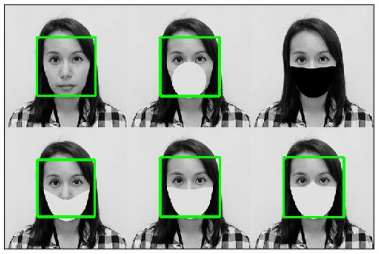
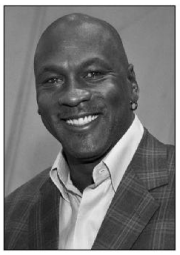
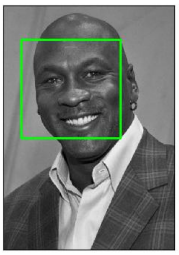
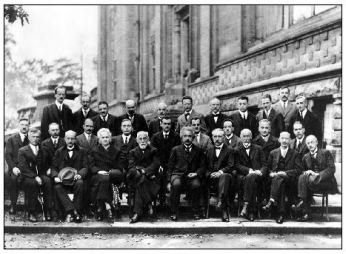
## D2. Praktikum Feature Matching dan Face Detection

8. Lakukan Face Detection untuk image object lain yang tersedia pada (/images/facedet). Tampilkan seperti pada contoh berikut.



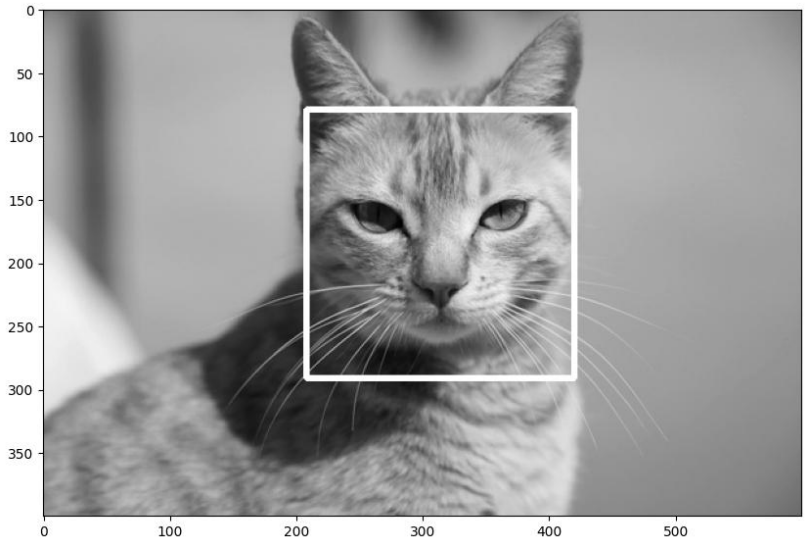



Perhatikan pada hasil face detection diatas. Secara keseluruhan, face detection dapat

dilakukan dengan baik, bahkan untuk image berupa gambar bukan foto, wajah bermasker, atau wajah yang berukuran kecil (solvay).

9. Pada soal sebelumnya, wajah kucing tidak bisa dideteksi dengan baik. Lakukan deteksi wajah kucing hingga muncul rectangle pada bagian wajah kucing. Petunjuk pada soal ini, perhatikan pretrained features yang telah disediakan OpenCV. Gunakan xml yang ada jika memang telah disediakan. Jika belum ada, coba cari dengan searching melalui search engines.



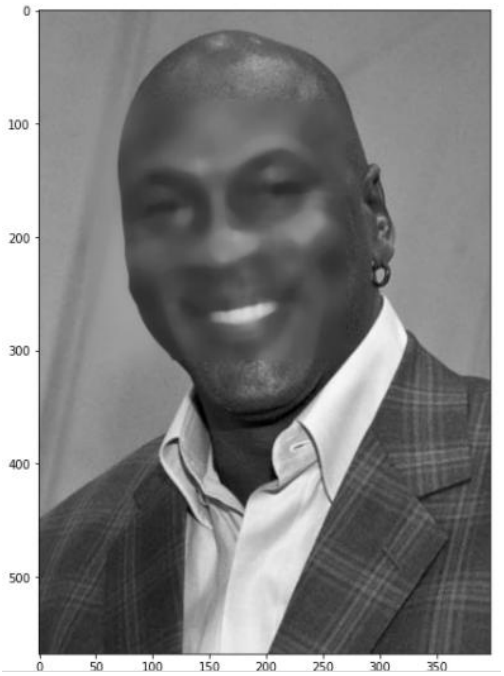

10. Cobakan juga untuk eyes detection pada wajah Pak Prabowo:




11. Lakukan deteksi senyuman pada gambar people:



12. Lakukan Face Tracking menggunakan Google Colab. Petunjuk, Tutorial lengkap tentang akses kamera dan FaceDetection pada google colab dapat dilihat di link berikut: https://www.youtube.com/watch?v=YjWh7QvVH60
13. Lakukan Blurring pada bagian wajah yang terdeteksi. Berikut contoh keluaran yang diharapkan. Petunjuk: anda dapat menggunakan cv.medianBlur untuk melakukan Blurring




14. Mengolah gambar KTM masing-masing
    a. Berikut ini contoh KTM




    b. Tampilkan Foto KTM seperti contoh berikut ini:

FOTO KTM



c. Lakukan Deteksi Obyek KTM pada gambar yang berisi KTM bersama barang lain dengan menggunakan **Konsep Feature Matching**

### D3. Tugas Pengayaan : Pengenalan Karakter KTM dengan CNN

Tampilkan karakter-karakter yang ada pada KTM Anda dengan menggunakan deep learning untuk melakukan pengenalan karakter. Sebelum melakukan training data, terlebih dahulu di siapkan data yang akan dilakukan training yaitu data image nomor angka 0-9. Untuk membuat data tersebut bisa gunakan image editor untuk dilakukan croping satu persatu. Kemudian gunakan source code di bawah ini untuk proses persiapan untuk training.

```
import os
import cv2
import random
import numpy as np
import pickle
from tqdm import tqdm

DATADIR = "dataset/training"      # satu subfolder per karakter: 0..9, A..Z
width, height = 100, 100

dirs = sorted(os.listdir(DATADIR))        # daftar nama kelas
training_data = []

for char_name in dirs:
    path = os.path.join(DATADIR, char_name)
    class_number = dirs.index(char_name)
    for img in tqdm(os.listdir(path), desc=char_name):
        try:
            img_array = cv2.imread(os.path.join(path, img),
                                   cv2.IMREAD_GRAYSCALE)
            new_array = cv2.resize(img_array, (width, height))
            training_data.append([new_array, class_number])
        except Exception:
            pass

random.shuffle(training_data)

X, Y = [], []
for feature, label in training_data:
    X.append(feature)
    Y.append(label)

X = np.array(X).reshape(-1, width, height, 1)
Y = np.array(Y)

pickle.dump(X,    open("X.pickle", "wb"))
pickle.dump(Y,    open("Y.pickle", "wb"))
pickle.dump(dirs, open("kelas.pickle", "wb"))

print("jumlah data :", len(training_data))
print("jumlah kelas:", len(dirs), dirs)
```

Penjelasan kode di atas adalah sebagai berikut ini

i. Import terlebih dahulu beberapa paket yang dibutuhkan, ada beberapa paket yang baru misalkan tqdm digunakan untuk meload data diikuti dengan progress bar, numpy merupakan sebuah paket yang digunakan untuk melakukan operasi-operasi matriks atau array serta pickle adalah depedensi untuk menyimpan file untuk model data training.
ii. Looping data training yang kelak digunakan untuk melabeli hasil pengenalan. Looping semua file training untuk diubah ke dalam sebuah image array.
iii. Variabel X dan variabel Y digunakan untuk menyimpan label dan feature, label berisi karakter A-Z dan 0-9 sedangkan feture berisi data image array masing-masing label tersebut.
iv. Terakhir tulis isi variabel X dan variabel Y ke dalam sebuah file pickle. File pickle tersebut nanti akan diload ketika proses training data.

Lakukan Training Data

Code berikut digunakan untuk melakukan training data

```
import pickle
from keras.layers import (Input, Conv2D, MaxPooling2D,
                          Flatten, Dense, ZeroPadding2D)
from keras.models import Model
from keras.optimizers import Adam
from keras.utils import to_categorical

X = pickle.load(open("X.pickle", "rb"))
Y = pickle.load(open("Y.pickle", "rb"))

Y = to_categorical(Y)
X = X / 255.0
width, height = 100, 100
jumlah_kelas = Y.shape[1]        # diambil dari data, tidak ditulis tetap

inputs     = Input(shape=(width, height, 1))
conv_layer = ZeroPadding2D(padding=(2, 2))(inputs)
conv_layer = Conv2D(16, (5, 5), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = MaxPooling2D((2, 2))(conv_layer)
conv_layer = Conv2D(32, (3, 3), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = Conv2D(32, (3, 3), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = MaxPooling2D((2, 2))(conv_layer)
conv_layer = Conv2D(64, (3, 3), strides=(1, 1), activation='relu')(conv_layer)

flaten   = Flatten()(conv_layer)
fc_layer = Dense(256, activation='relu')(flaten)
fc_layer = Dense(64,  activation='relu')(fc_layer)
outputs  = Dense(jumlah_kelas, activation='softmax')(fc_layer)

model = Model(inputs=inputs, outputs=outputs)
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

riwayat = model.fit(X, Y, epochs=20, verbose=1, validation_split=0.2)
model.save('anpr.keras')

print("akurasi training  :", riwayat.history['accuracy'][-1])
print("akurasi validasi  :", riwayat.history['val_accuracy'][-1])
```

Dengan kode di atas, kita akan membuat sebuah model, kode di atas menggunakan deep learning dengan arsitektur CNN(Convolutional Neural Network). Perbedaan mendasar antara arsitektur ini dan neural network biasa terletak pada feature extraction. Feature adalah ciri khas yang membedakan satu objek dengan objek lain. Sebagai contoh karakter A dan karakter B pada pelat kendaraan memiliki ciri bentuk yang berbeda. Feature extraction yang dimiliki CNN(Convolutional Neural Network) sudah disediakan, kita tinggal mengatur arsitektur dan parameter di dalam model. Sedangkan pada neural network biasa, kita harus mencari sendiri atau melakukannya secara manual untuk mencari feature tersebut. Di bawah ini adalah bagian kode yang digunakan untuk membuat atau membangun sebuah feature tersebut.

```
conv_layer = ZeroPadding2D(padding=(2, 2))(inputs)
conv_layer = Conv2D(16, (5, 5), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = MaxPooling2D((2, 2))(conv_layer)
conv_layer = Conv2D(32, (3, 3), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = Conv2D(32, (3, 3), strides=(1, 1), activation='relu')(conv_layer)
conv_layer = MaxPooling2D((2, 2))(conv_layer)
conv_layer = Conv2D(64, (3, 3), strides=(1, 1), activation='relu')(conv_layer)
```

## Testing Data

Contoh kode berikut digunakan untuk melakukan testing pengenalan karakter

```
import os
import cv2
import pickle
import numpy as np
import tensorflow as tf

data_dir_testing = "dataset/testing"
width, height = 100, 100

model = tf.keras.models.load_model("anpr.keras")
dirs  = pickle.load(open("kelas.pickle", "rb"))   # urutan kelas saat training

for car in sorted(os.listdir(data_dir_testing)):
    temp = ""
    folder = os.path.join(data_dir_testing, car)
    for char_img in sorted(os.listdir(folder)):
        img_array = cv2.imread(os.path.join(folder, char_img),
                               cv2.IMREAD_GRAYSCALE)
        new_array = cv2.resize(img_array, (width, height))
        new_array = new_array.reshape(-1, width, height, 1) / 255.0
        prediction = model.predict(new_array, verbose=0)
        temp += dirs[np.argmax(prediction[0])]
    print("folder name: {} no: {}".format(car, temp))
```

--- SEMANGAT BELAJAR ---